Raspberry Pi — Full Hardened Setup Guide

> Image: `2026-04-21-raspios-trixie-arm64-lite.img` (Debian 13 Trixie, 64-bit Lite) Covers: Format SD → Flash → SSH → WiFi → Static IP → SSH Key Auth → MAC Spoof → WireGuard Vim is used throughout. No GUI assumed. Headless from the start.

OS Stack — Trixie specifics:

- First boot: cloud-init only (legacy `firstrun.sh` removed)
- Networking: Netplan → NetworkManager (not dhcpcd)
- Boot partition: `/boot/firmware/` (not `/boot/`)
- Imager: v2.0.6+ required — older versions fail silently on Trixie
- Hostname: must be changed via `user-data` — `raspi-config` changes are overwritten by cloud-init on reboot

---

PHASE 0 — Prerequisites

What you need:

- Raspberry Pi 4/5
- MicroSD card (16GB+ recommended)
- Linux/macOS workstation
- Ethernet cable (recommended for first boot) OR WiFi credentials

---

PHASE 1 — Prepare & Format the SD Card

Before flashing, identify and format the SD card. Always confirm your device path with `lsblk` — getting this wrong overwrites the wrong drive.

```
        lsblk -o NAME,SIZE,TYPE,MOUNTPOINT  # device, size, type, mount point
        lsblk -f                            # filesystem, label, UUID
```

Note your values:

```
Device:      /dev/sdX        ← e.g. /dev/sdb
Partition:   /dev/sdX1       ← e.g. /dev/sdb1
Mount point: /media/...      ← exact path from MOUNTPOINT column
```

> Mount paths typically follow `/media/<username>/<UUID>` on Ubuntu/Debian but vary by distro, user, and card UUID. Never assume — always confirm with `lsblk` on your current machine before running any commands.

Option A: RPi Imager — Erase/Format

1. Open RPi Imager: `rpi-imager`
2. Click **Choose OS** → scroll to bottom → select **Erase (Format card as FAT32)**
3. Click **Choose Storage** → select your SD card (confirm by size)
4. Click **Write** → Confirm

> This wipes the card and formats it as FAT32 with a clean MBR partition table. Equivalent to the command method below.

Option B: Command Line — Format to FAT32

> Replace `/dev/sdX` with your actual device from `lsblk` above. Do not copy these commands blindly — confirm your device path first.

```
        lsblk -f                                                          # confirm /dev/sdX
        sudo umount /dev/sdX[1-9] 2>/dev/null                             # unmount ALL partitions
        sudo wipefs -a /dev/sdX                                           # wipe existing FS signatures
        sudo parted /dev/sdX --script mklabel msdos mkpart primary fat32 1MiB 100% set 1 boot on
        sudo partprobe /dev/sdX                                           # kernel reread partition table
        sudo mkfs.vfat -F 32 -n "SDCARD" /dev/sdX1                        # format FAT32
        lsblk -f /dev/sdX                                                 # verify
```

Expected output after formatting:

```
NAME   FSTYPE LABEL  UUID
sdX
└─sdX1 vfat   SDCARD XXXX-XXXX
```

Step	What it does
`umount`	Releases OS lock on the partition
`mklabel msdos`	Creates MBR partition table (Pi compatible)
`mkpart primary fat32 1MiB 100%`	Single partition using full card
`set 1 boot on`	Marks partition as bootable
`mkfs.vfat -F 32`	Writes FAT32 filesystem
`-n "SDCARD"`	Labels the volume

> `msdos` is the partition table type (MBR), not the filesystem. FAT32 is applied by `mkfs.vfat`. RPi Imager uses this exact combination for compatibility across all Pi models.

SD card is now clean and ready to flash.

PHASE 2 — Flash the OS

Option A: Raspberry Pi Imager — Notes Only

> Assumes proficiency with Imager UI. This section flags what Imager handles automatically that the manual `dd` path requires you to do yourself.
> **Imager v2.0.6+ required for Trixie.** Older versions fail silently. Check version: Help → About. UI shortcut: Ctrl+Shift+X for OS Customisation.

What Imager does automatically that manual `dd` does not:

Task	Imager	Manual `dd`
Write `user-data`	✅ Auto	Phase 4 Step 3
Write `network-config`	✅ Auto	Phase 4 Step 5
Set `instance-id`	✅ Auto	Inherited from image
Modify `cmdline.txt`	✅ Auto	Phase 4 Step 4
Password hash format	✅ Yescrypt	SHA-512 fine
Locale/timezone/keyboard	✅ UI-driven	Phase 4 Step 3

> **Critical caveat:** Imager 2.x with a **locally downloaded `.img` file** assumes `init_format: none` and silently SKIPS OS customisation — hostname, SSH, WiFi will NOT apply. Use manual `dd` path for local images. Imager customisation only works correctly with images from the catalog.

Option B: `dd` Command (Full Control)

```
# 1. Download Raspberry Pi OS Lite image
# https://www.raspberrypi.com/software/operating-systems/
# Current image: 2026-04-21-raspios-trixie-arm64-lite.img.xz
# Decompress
unxz 2026-04-21-raspios-trixie-arm64-lite.img.xz

# 2. Find your SD card device (DO NOT get this wrong)
lsblk
# Look for your SD card — e.g., /dev/sdb or /dev/mmcblk0

# 3. Unmount if auto-mounted
sudo umount /dev/sdX1 2>/dev/null
sudo umount /dev/sdX2 2>/dev/null

# 4. Write image — replace /dev/sdX with your actual device from lsblk
sudo dd \
  if=2026-04-21-raspios-trixie-arm64-lite.img \
  of=/dev/sdX \
  bs=4M \
  status=progress \
  conv=fsync

# 5. Sync and eject
sync
sudo eject /dev/sdX
```

> `bs=4M` sets block size for speed. `conv=fsync` flushes buffers before exit. Never use `/dev/sdX` without confirming with `lsblk` first. After ejecting, proceed to Phase 3 to mount and configure the SD card.

PHASE 3 — Mount SD Card & Configure Boot Files

After ejecting, reinsert the SD card. Your desktop may auto-mount both partitions. Confirm mount paths before editing anything:

```
lsblk -o NAME,LABEL,MOUNTPOINT                  # confirm mount paths
sudo mount /dev/sdX1 /media/$USER/bootfs        # bootfs (if not auto-mounted)
sudo mount /dev/sdX2 /media/$USER/rootfs        # rootfs (if not auto-mounted)
```

> Always confirm with `lsblk` before mounting — double-mounting causes conflicts. Trixie boot partition label: `bootfs` Use exact paths from `lsblk` — never assume.

PHASE 4 — Pre-Boot Checklist: Configure SD Card Before First Boot

What Imager 2.x writes — forensic findings:

```
bootfs:  user-data ✅ · network-config ✅ · meta-data ✅ · cmdline.txt ✅ (modified)
         ssh ❌ · firstrun.sh ❌ · wpa_supplicant.conf ❌
rootfs:  preconfigured.nmconnection ❌ · rfkill wlan files ❌ (don't exist pre-boot)
```

First boot flow: `cmdline.txt` triggers cloud-init → `user-data` creates user/SSH/packages → `network-config` → netplan → NM → WiFi.

> `lock_passwd: false` REQUIRED in `users:` — default `true` silently locks account. Hostname only via `user-data` — `raspi-config` is overwritten on reboot. Packages install via `runcmd` (not `packages:` block) — needs NTP sync first on Trixie because `sqv` (modern GPG verifier) rejects future-dated signatures when the Pi clock is wrong on first boot. `ufw` and `iptables-persistent` conflict — pick one. Guide uses `ufw`. Save iptables rules manually with `iptables-save > /etc/iptables/rules.v4`.

Step 1 — Generate Password Hash

Do this on your workstation before editing any files:

```
openssl passwd -6          # -6 = SHA-512 (strongest available)
# Enter and confirm your desired password when prompted
# Copy the full output — starts with $6$...
```

> Single quotes ONLY when pasting the hash into any file. Double quotes cause bash to expand `$6$` as a variable and corrupt the hash.

Step 2 — Generate WiFi PSK Hash

```
# Generates a hashed PSK from your SSID and password
wpa_passphrase "YourSSID" "YourPassword" | grep -v "#psk" | grep psk
# Output: psk=8f1f372af602...  (64 char hex — copy this value only, no quotes)
```

> Using a hashed PSK instead of plaintext password is more secure. The hash is SSID-specific — regenerate if you change SSIDs. If WiFi fails with the hash, fall back to plaintext password in double quotes.

Step 3 — Edit `user-data`

This file controls hostname, user creation, password, and SSH.

> **Note on SSH:** `enable_ssh: true` and `runcmd systemctl enable --now ssh` in this file handle SSH on Trixie. The legacy `ssh` sentinel file is NOT needed — Imager 2.x does not create it on Trixie.

```
sudo vim /media/$USER/bootfs/user-data
```

Replace the entire contents with:

> First line MUST be exactly `#cloud-config` — no spaces, no BOM. Any deviation causes cloud-init to silently ignore the entire file.

```
#cloud-config
manage_resolv_conf: false

# Hostname — only change via user-data on Trixie
# raspi-config changes are overwritten by cloud-init on reboot
hostname: <your-hostname>
manage_etc_hosts: true
preserve_hostname: false

timezone: America/New_York
keyboard:
  model: pc105
  layout: "us"

# Pi has no RTC — apt fails before NTP sync without this
apt:
  preserve_sources_list: true
  conf: |
    Acquire {
      Check-Date "false";
    };

packages:
  - avahi-daemon
  - vim
  - wireguard
  - macchanger
  - watchdog
  - ufw                # firewall (conflicts with iptables-persistent — pick one)
  - fail2ban
  - iptables
  - nmap
  - net-tools
  - git
  - unzip
  - tar
  - jq
  - tree
  - tmux

users:
  - name: pi
    groups: users,adm,dialout,audio,netdev,video,plugdev,cdrom,games,input,gpio,spi,i2c,render,sudo
    shell: /bin/bash
    lock_passwd: false     # REQUIRED — default is true which locks account even with hash set
    passwd: '<paste-$6$-hash-here>'

# enable_ssh is a cloudinit-rpi extension — works on Pi OS Trixie only
enable_ssh: true
ssh_pwauth: true

# Pi-specific interfaces (cloudinit-rpi module)
rpi:
  interfaces:
    serial: true

write_files:
  # MAC spoof via systemd-networkd .link file
  # Applied by udev when wlan0 is first detected — BEFORE NetworkManager
  # This is the only way to spoof MAC before WiFi association
  - path: /etc/systemd/network/00-wlan0-mac.link
    permissions: '0644'
    owner: root:root
    content: |
      [Match]
      OriginalName=wlan0

      [Link]
      MACAddress=D2:AD:B3:3F:BE:EF
      NamePolicy=keep kernel database onboard slot path

  # Belt-and-suspenders backup service if .link file doesn't apply
  - path: /etc/systemd/system/macspoof-wlan0.service
    permissions: '0644'
    owner: root:root
    content: |
      [Unit]
```

```
            Description=MAC spoof backup (if .link file failed)
            Wants=network-pre.target
            Before=network-pre.target NetworkManager.service
            After=sys-subsystem-net-devices-wlan0.device
            BindsTo=sys-subsystem-net-devices-wlan0.device
            DefaultDependencies=no
            ConditionFileNotEmpty=!/sys/class/net/wlan0/address

            [Service]
            Type=oneshot
            RemainAfterExit=yes
            ExecStartPre=/usr/sbin/ip link set dev wlan0 down
            ExecStart=/usr/bin/macchanger --mac=D2:AD:B3:3F:BE:EF wlan0
            ExecStartPost=/usr/sbin/ip link set dev wlan0 up

            [Install]
            WantedBy=network-pre.target

        runcmd:
          # CRITICAL: Pi has no RTC. On first boot it thinks it's the image build date.
          # Modern apt uses sqv (Sequoia GPG) which rejects signatures dated in the future.
          # Force NTP sync BEFORE package install or apt fails with "Not live until..." errors.
          - [ sh, -c, "timedatectl set-ntp true" ]
          - [ sh, -c, "until timedatectl status | grep -q 'System clock synchronized: yes'; do sleep 2; done" ]
          # Now safe to install packages
          - [ sh, -c, "apt-get update && apt-get install -y avahi-daemon vim wireguard macchanger watchdog ufw fail2ban
iptables nmap net-tools git unzip tar jq tree tmux" ]
          # crda deprecated on Trixie — cmdline.txt handles regdomain — sed harmless if absent
          - [ sh, -c, "sed -i 's/^[[:space:]]*REGDOMAIN=.*/REGDOMAIN=US/' /etc/default/crda 2>/dev/null || true" ]
          - [ systemctl, daemon-reload ]
          - [ systemctl, enable, macspoof-wlan0.service ]
          - [ sh, -c, "systemctl enable --now ssh || true" ]
```

Field	Why it matters
`passwd` in single quotes	Double quotes corrupt `$6$` hash
`lock_passwd: false`	**Required** — default `true` silently locks account
`hostname`	Only reliable hostname method on Trixie
`enable_ssh: true`	cloudinit-rpi extension — Pi OS only
`ssh_pwauth: true`	Enables password auth in sshd
`apt Check-Date "false"`	Pi has no RTC — apt fails before NTP without this
`.link` file (00-wlan0-mac.link)	Applied by udev at device detection — runs BEFORE NetworkManager touches wlan0
`BindsTo=wlan0.device` (backup service)	Prevents backup service firing before wlan0 exists
SHA-512 (`$6$`) hash	Imager uses yescrypt (`$y$`) — both work on Trixie
No NOPASSWD	sudo prompts for password — intentional for hardened node

Step 4 — Verify `cmdline.txt`

```
        # Verify both args present (must return matches)
        grep -oE 'cfg80211.ieee80211_regdom=US|ds=nocloud' /media/$USER/bootfs/cmdline.txt
```

If either missing, edit and append to end of single line:

```
        vim /media/$USER/bootfs/cmdline.txt
        # In vim:
        #   A    → jump to end of line, enter Insert mode
        #          type space then: cfg80211.ieee80211_regdom=US ds=nocloud;i=rpi-manual-1
        #   Esc
        #   :wq

        wc -l /media/$USER/bootfs/cmdline.txt   # must show 1 (single line)
```

`cfg80211.ieee80211_regdom=US` — sets WiFi regulatory domain at kernel level before userspace. This is the primary regdomain mechanism on Trixie — crda is deprecated. `ds=nocloud;i=<instance-id>` — tells cloud-init to use NoCloud datasource and sets the instance-id. The `;` is real cloud-init syntax — do not escape it. `cmdline.txt` MUST remain a single line — no line breaks, no trailing newline.

Step 5 — Edit `network-config`

This file controls WiFi and IP assignment.

```
sudo vim /media/$USER/bootfs/network-config
```

Option A — DHCP (default, IP changes per boot):

```
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    eth0:
      dhcp4: true
      dhcp6: true
      optional: true
  wifis:
    wlan0:
      dhcp4: true
      optional: true
      regulatory-domain: "US"
      access-points:
        "<YourSSID>":
          password: "<paste-64-char-psk-hash-here>"
```

Option B — Static IP (fixed, recommended for WireGuard):

> Pick an IP OUTSIDE your router's DHCP pool to avoid conflicts. Check router admin for pool range — typical default is `.100-.200`. Pick something below `.100` or above `.200`.

```
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    eth0:
      dhcp4: true
      dhcp6: true
      optional: true
  wifis:
    wlan0:
      dhcp4: false                    # disable DHCP
      optional: true
      regulatory-domain: "US"
      addresses:
        - 192.168.1.50/24             # static IP — outside DHCP pool
      gateway4: 192.168.1.1            # your router IP
      nameservers:
        addresses: [1.1.1.1, 9.9.9.9]
      access-points:
        "<YourSSID>":
          password: "<paste-64-char-psk-hash-here>"
```

Belt-and-suspenders — also add DHCP reservation in router:

```
Router admin → DHCP → Reservations
MAC: D2:AD:B3:3F:BE:EF (the spoofed MAC from user-data)
IP:  192.168.1.50
```

Pi gets the static IP from `network-config` immediately. If that ever fails, the router reservation hands it `.50` anyway. No conflicts possible since `.50` is outside the DHCP pool.

> `renderer: NetworkManager` — Imager omits this because the system default is already NM on Trixie. Including it is safe and self-documenting. If included, it MUST be at the top level next to `version:` — NEVER inside `wifis:` or any interface block. Wrong placement = `Invalid network-config` error, WiFi silently fails. `regulatory-domain: "US"` must be sibling of `access-points:` under `wlan0:` — not inside it. `optional: true` on both interfaces — without this, boot hangs 2 minutes waiting for carrier if either interface is down. cloud-init writes this to `/etc/netplan/50-cloud-init.yaml` on first boot. netplan then generates NM keyfiles at `/run/NetworkManager/system-connections/`. If WiFi fails with 64-char PSK hash, replace with plaintext in double quotes.

Step 6 — Write `preconfigured.nmconnection` (Optional)

> **Forensic finding: Imager 2.x with cloudinit-rpi does NOT write this file.** Imager touches ZERO files on rootfs. WiFi works via cloud-init reading `network-config` → netplan → NM keyfiles generated at first boot. This step is an OPTIONAL fallback — useful if cloud-init fails or you want a static NM connection that works independently of cloud-init.

> rootfs already mounted in Phase 3 — use the same path:

```
sudo vim /media/$USER/rootfs/etc/NetworkManager/system-connections/preconfigured.nmconnection
```

File contents:

```
[connection]
id=preconfigured
type=wifi
autoconnect=true

[wifi]
mode=infrastructure
ssid=<YourSSID>
hidden=false

[wifi-security]
auth-alg=open
key-mgmt=wpa-psk
psk=<paste-64-char-psk-hash-here>

[ipv4]
method=auto

[ipv6]
addr-gen-mode=default
method=auto

[proxy]
```

```
# Critical — must be 600 root owned or NetworkManager ignores it
sudo chmod 600 /media/$USER/rootfs/etc/NetworkManager/system-connections/preconfigured.nmconnection
sudo chown root:root /media/$USER/rootfs/etc/NetworkManager/system-connections/preconfigured.nmconnection

# Verify permissions
ls -la /media/$USER/rootfs/etc/NetworkManager/system-connections/
# Should show: -rw------- root root preconfigured.nmconnection
```

Step 7 — rfkill Check

> **Forensic finding: wlan rfkill files do NOT exist pre-boot.** They are auto-created post-boot when the `brcmfmac` WiFi driver registers. Pre-writing them before first boot does nothing — they are orphan files. WiFi radio is handled by `cfg80211.ieee80211_regdom=US` in `cmdline.txt` and `regulatory-domain: "US"` in `network-config`. Only intervene if you have a known-blocked image.

```
# Check existing rfkill state (bluetooth files only on fresh image)
ls /media/$USER/rootfs/var/lib/systemd/rfkill/
# Each file should contain 0 (unblocked)
# Content of 1 = blocked — fix if found:
for f in /media/$USER/rootfs/var/lib/systemd/rfkill/*; do
    echo -n "$(basename $f): " && cat "$f"
done
# If any show 1, fix: sudo sh -c "echo 0 > /media/$USER/rootfs/var/lib/systemd/rfkill/<filename>"

# Unmount rootfs
sudo umount /media/$USER/rootfs
```

Step 8 — Verify All Files Before Ejecting

```
# 1. View all bootfs file contents
cat /media/$USER/bootfs/user-data        # hostname, lock_passwd: false, hash, packages, runcmd
cat /media/$USER/bootfs/network-config   # SSID, renderer, regulatory-domain, optional: true
cat /media/$USER/bootfs/meta-data        # confirm instance-id present
cat /media/$USER/bootfs/cmdline.txt      # cfg80211.ieee80211_regdom=US and ds=nocloud on single line

# 2. If preconfigured.nmconnection was written — needs sudo (file is 600 root)
sudo cat /media/$USER/rootfs/etc/NetworkManager/system-connections/preconfigured.nmconnection

# 3. Verify perms on preconfigured.nmconnection (if used)
ls -la /media/$USER/rootfs/etc/NetworkManager/system-connections/ 2>/dev/null
# Must show: -rw------- root root preconfigured.nmconnection

# 4. YAML validation — catches tab/indentation errors invisible to cat
python3 -c "import yaml; yaml.safe_load(open('/media/$USER/bootfs/user-data'))" && echo 'user-data OK'
python3 -c "import yaml; yaml.safe_load(open('/media/$USER/bootfs/network-config'))" && echo 'network-config OK'

python3 -c "import yaml; yaml.safe_load(open('/media/$USER/bootfs/meta-data'))" && echo 'meta-data OK'

# 5. Verify cmdline.txt is single line with required args
grep -o 'cfg80211.ieee80211_regdom=US' /media/$USER/bootfs/cmdline.txt && echo 'regdomain OK'
grep -o 'ds=nocloud' /media/$USER/bootfs/cmdline.txt && echo 'datasource OK'
wc -l /media/$USER/bootfs/cmdline.txt   # must show 1
```

Step 9 — Unmount & Eject

```
# Unmount both partitions
sudo umount /dev/sdX1
sudo umount /dev/sdX2

# Confirm via kernel — definitive ground truth
cat /proc/mounts | grep sdX   # empty output = unmounted

# Confirm via lsblk — MOUNTPOINT column must be blank
lsblk -o NAME,LABEL,MOUNTPOINT /dev/sdX
```

Only then physically remove the SD card and insert into the Pi.

> File manager may show stale cached view of the card after unmount — ignore it. `cat /proc/mounts` is kernel-level truth.

> `meta-data` is required — without it cloud-init refuses to read `user-data`. Imager sets `instance-id: rpi-imager-<unix-millis>`. Any unique string works. To force cloud-init to fully re-run on a previously booted image: change the `instance-id` here AND wipe `/var/lib/cloud/instances/` on rootfs. `enable_ssh: true` + `runcmd systemctl enable --now ssh` handle SSH on Trixie. The `ssh` sentinel file is NOT created by Imager 2.x — skip it on Trixie. `wpa_supplicant.conf`, `userconf.txt`, `hostname` files are legacy — do not use.

PHASE 5 — First Boot & Find the Pi

Insert SD into Pi, power on, wait **3-5 minutes** for first boot. First boot installs all packages from `user-data` — takes longer than subsequent boots.

Option 1 — Check Router DHCP Lease Table (Easiest)

Most routers expose leases via their admin page, but you can also query from the command line if your router supports it or you're running a local DHCP server:

```
# Show all devices recently seen on your subnet
arp -a
# Note: hostnames may not resolve here — most home networks show IP/MAC only
# Compare to your pre-boot state to spot the new Pi
```

> If your router doesn't expose leases via CLI, log into the admin panel (typically `192.168.1.1` or `192.168.0.1`) and look for a **DHCP Clients** or **Connected Devices** table. The Pi will appear as `<your-hostname>` or `Raspberry Pi Foundation`.

Option 2 — Scan Your Subnet (nmap)

```
# One-shot scan: MAC + ports + OS detection (adjust subnet)
sudo nmap -O -p 22,80,443 192.168.1.0/24 --open

# Filter for spoofed MAC if present
sudo nmap -sn 192.168.1.0/24 | grep -B2 -i "BE:EF"        # stable across D2/D3 spoof
```

Flags: `-sn` ping only · `-O` OS detect · `-p` port(s) · `--open` open hosts only · `-A` aggressive (-O -sV -sC –traceroute)

Option 3 — Try Hostname (mDNS)

```
ssh pi@<your-hostname>.local
```

What First Boot Does — rootfs Changes

Once cloud-init runs on first boot, the following are materialized on the root filesystem automatically. Use these to verify everything applied correctly after you connect:

```
# Run all verification checks at once
hostname && cat /etc/hostname                    # 1. hostname set
cat /etc/hosts | grep 127.0.1.1                  # 2. /etc/hosts updated
sudo systemctl is-active ssh                     # 3. SSH running
sudo systemctl is-enabled ssh                    # 4. SSH enabled at boot
ls /run/NetworkManager/system-connections/       # 5. NM WiFi profile
ls /var/lib/cloud/instances/                     # 6. cloud-init ran
sudo systemctl is-active avahi-daemon            # 7. avahi running
id pi                                            # 8. pi user exists
ip link show wlan0 | grep ether                  # 9. MAC ends in BE:EF (D2 or D3 prefix)
```

Expected state after successful first boot:

Component	Expected State
`/etc/hostname`	Matches `hostname:` in `user-data`
`/etc/hosts`	`127.0.1.1` entry matches hostname
`ssh.service`	Enabled + active
SSH host keys	Present under `/etc/ssh/ssh_host_*_key`
NM WiFi profile	File in `/run/NetworkManager/system-connections/`
`/var/lib/cloud/`	Instance directory present
`avahi-daemon`	Active if listed in `packages:`
`pi` user	Created with hashed password from `user-data`

One-line summary: `bootfs` seeds the intent via `network-config` + `user-data`. `rootfs` first boot materializes that intent into hostname, pi account, SSH host keys + enabled `ssh.service`, NM WiFi profile, optional Avahi, and `/var/lib/cloud/` state proving cloud-init ran.

PHASE 6 — Static IP — Verify

Static IP configured pre-boot in Phase 4 Step 5 Option B. This phase verifies it applied correctly.

```
ssh pi@192.168.1.50                          # connect via static IP
ip addr show wlan0 | grep inet               # expect: 192.168.1.50/24
ip route | grep default                      # expect: default via 192.168.1.1
cat /etc/resolv.conf | grep nameserver       # expect: 1.1.1.1 9.9.9.9
# Router admin → DHCP Reservations should show spoofed MAC (ends BE:EF)
```

If IP doesn't match `network-config` — cloud-init may have cached the old config. Force reapply: `sudo cloud-init clean --reboot`

PHASE 7 — SSH Key Auth & Hardening

Step 1: Generate SSH Key Pair (on your workstation, NOT the Pi)

```
# On your laptop/workstation
ssh-keygen -t ed25519 -C "<your-hostname>-key" -f ~/.ssh/rpi_ed25519
# Press Enter for no passphrase, or set one for added security

# Two files created:
# ~/.ssh/rpi_ed25519       → private key (never share)
# ~/.ssh/rpi_ed25519.pub   → public key (goes on Pi)
```

Step 2: Copy Public Key to Pi

```
ssh-copy-id -i ~/.ssh/rpi_ed25519.pub pi@192.168.1.50

# Manual alternative if ssh-copy-id unavailable:
cat ~/.ssh/rpi_ed25519.pub | ssh pi@192.168.1.50 "mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600
~/.ssh/authorized_keys"
```

Step 3: Test Key Auth Before Locking Down

```
ssh -i ~/.ssh/rpi_ed25519 pi@192.168.1.50
# Should connect WITHOUT asking for password
```

Step 4: Harden SSH Daemon Config

Use a drop-in config file — safer than editing `sshd_config` directly, survives package upgrades, easier to revert:

```
sudo tee /etc/ssh/sshd_config.d/99-hardening.conf > /dev/null << 'EOF'
Port 2222
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
X11Forwarding no
MaxAuthTries 3
LoginGraceTime 20
ClientAliveInterval 60
ClientAliveCountMax 3
AllowUsers pi
EOF

# Validate before reloading — catches syntax errors
sudo sshd -t && echo "Config OK"

# Reload SSH service
sudo systemctl reload ssh

# Test from workstation — KEEP current session open while testing
ssh -i ~/.ssh/rpi_ed25519 -p 2222 pi@192.168.1.50
```

> Do NOT close existing session until new session connects. If locked out, mount SD on workstation and remove the drop-in: `sudo rm /media/$USER/rootfs/etc/ssh/sshd_config.d/99-hardening.conf`

Step 5: Firewall Basics

> `ufw` already installed via `packages:` in `user-data` — no install needed.

```
# Allow only your SSH port
sudo ufw allow 2222/tcp

# Allow WireGuard (we'll configure this next)
sudo ufw allow 51820/udp

# Enable
sudo ufw enable
sudo ufw status
```

PHASE 8 — MAC Address Spoofing — Verify

Target MAC: `D2:AD:B3:3F:BE:EF` (locally administered, unicast)

> **Why this matters:** MAC randomization at the edge is one layer of metadata hygiene — defeats correlation of the device against the Broadcom OUI (which would identify it as a Raspberry Pi) in DHCP logs, WiFi auth logs, and packet captures.

MAC spoofing is fully automated via `user-data` and applies BEFORE WiFi associates:

- `.link` file at `/etc/systemd/network/00-wlan0-mac.link` — udev applies at device detection
- `macchanger` package installed via `packages:` (for backup service)
- Backup `macspoof-wlan0.service` runs only if `.link` file didn't apply

> The `.link` file is the primary mechanism — it runs at udev level when the kernel first detects wlan0, BEFORE NetworkManager or any userspace service can touch the interface. This is the earliest possible point to spoof MAC.

Verify MAC + .link file + persistence

```
ip link show wlan0 | grep ether                    # expect ends BE:EF
cat /etc/systemd/network/00-wlan0-mac.link         # confirm .link file present

sudo reboot                                         # test persistence
# After reconnect:
ip link show wlan0 | grep ether                    # must still end BE:EF
```

Troubleshooting

```
# Check what MAC udev assigned and why
sudo udevadm info /sys/class/net/wlan0 | grep -i mac

# Check if .link file was processed
sudo journalctl -b | grep -i "wlan0\|MACAddress"

# Backup service state (only fires if .link failed)
sudo systemctl status macspoof-wlan0.service
```

Note on MAC byte values

> Input `D3ADB33F` is 4 bytes — a MAC requires 6 bytes. Full address used: `D2:AD:B3:3F:BE:EF` — D2 is locally administered + unicast (cleaner than D3 multicast). For a cleaner unicast spoof, change first byte to `D2`: `D2:AD:B3:3F:BE:EF`. Either works for obfuscation. To change, edit `write_files` in `user-data`.

PHASE 9 — WireGuard VPN (Remote Access, No Phone)

WireGuard gives you an encrypted tunnel back into the Pi from anywhere. No QR code needed — we configure everything via files.

Architecture:

```
[Your Laptop] ←── WireGuard tunnel ──→ [RPi = WireGuard Server]
```

> This assumes the RPi is behind your home router. You'll need to forward UDP port 51820 on the router to `192.168.1.50`.

Step 1: WireGuard Already Installed

> `wireguard` already installed via `packages:` in `user-data`. Verify it's available:

```
which wg && which wg-quick && echo "WireGuard ready"
```

Step 2: Generate Server Keys (on the Pi)

```
# Generate and store keys with proper permissions
wg genkey | sudo tee /etc/wireguard/server_private.key | \
  wg pubkey | sudo tee /etc/wireguard/server_public.key

# Lock down private key
sudo chmod 600 /etc/wireguard/server_private.key

# View keys (you'll need these)
sudo cat /etc/wireguard/server_private.key
sudo cat /etc/wireguard/server_public.key
```

Step 3: Generate Client Keys (on the Pi, for your laptop)

```
wg genkey | sudo tee /etc/wireguard/client_private.key | \
  wg pubkey | sudo tee /etc/wireguard/client_public.key

sudo chmod 600 /etc/wireguard/client_private.key

sudo cat /etc/wireguard/client_private.key   # → copy for laptop config
sudo cat /etc/wireguard/client_public.key    # → paste into server config
```

Step 4: Create Server Config on Pi

```
sudo vim /etc/wireguard/wg0.conf
```

Server config (`/etc/wireguard/wg0.conf`):

```
[Interface]
Address = 10.0.0.1/24
ListenPort = 51820
PrivateKey = <PASTE server_private.key content here>

# Enable IP forwarding for routing (optional but useful)
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -t nat -A POSTROUTING -o wlan0 -j MASQUERADE
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -t nat -D POSTROUTING -o wlan0 -j MASQUERADE

# Laptop peer
[Peer]
PublicKey = <PASTE client_public.key content here>
AllowedIPs = 10.0.0.2/32
```

```
# Save and quit (Esc :wq), then lock down config
sudo chmod 600 /etc/wireguard/wg0.conf
```

Step 5: Enable IP Forwarding

> Trixie deprecates `/etc/sysctl.conf` — systemd-sysctl doesn't read it. Use a drop-in file in `/etc/sysctl.d/` instead.

```
echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-wireguard.conf
sudo sysctl --system                  # apply all sysctl.d files
sysctl net.ipv4.ip_forward            # verify — expect: net.ipv4.ip_forward = 1
```

Step 6: Start WireGuard on Pi

```
# Start and enable on boot
sudo systemctl enable wg-quick@wg0
sudo systemctl start wg-quick@wg0

# Check status
sudo systemctl status wg-quick@wg0
sudo wg show
```

Step 7: Create Client Config (for your Laptop)

On your laptop, create the WireGuard config file manually — no QR code needed:

```
# Linux laptop
sudo apt install wireguard -y
sudo mkdir -p /etc/wireguard
sudo vim /etc/wireguard/rpi-tunnel.conf
```

Client config (`rpi-tunnel.conf`):

```
[Interface]
Address = 10.0.0.2/24
PrivateKey = <PASTE client_private.key content here>
DNS = 1.1.1.1

[Peer]
PublicKey = <PASTE server_public.key content here>
Endpoint = <YOUR_HOME_PUBLIC_IP>:51820
AllowedIPs = 10.0.0.1/32
PersistentKeepalive = 25
```

`AllowedIPs = 10.0.0.1/32` → only tunnel traffic to the Pi (split tunnel). Use `AllowedIPs = 0.0.0.0/0` to route ALL traffic through the Pi.

```
        # Secure the file
        sudo chmod 600 /etc/wireguard/rpi-tunnel.conf
```

Step 8: Connect from Laptop

```
        # Bring up tunnel
        sudo wg-quick up rpi-tunnel

        # Verify connection
        ping 10.0.0.1               # ping the Pi's WireGuard IP

        # SSH through the tunnel
        ssh -i ~/.ssh/rpi_ed25519 -p 2222 pi@10.0.0.1

        # Check tunnel status
        sudo wg show

        # Tear down when done
        sudo wg-quick down rpi-tunnel
```

Step 9: Router Port Forward

On your home router admin panel:

```
Protocol:      UDP
External Port: 51820
Internal IP:   192.168.1.50
Internal Port: 51820
```

Find your public IP:

```
        curl ifconfig.me
```

PHASE 10 — Final System Hardening

```
        # Update everything
        sudo apt update && sudo apt upgrade -y

        # Disable unnecessary services — DO NOT disable avahi-daemon
        # (avahi powers .local hostname resolution — keep it enabled)
        sudo systemctl disable bluetooth

        # Set timezone (already set via user-data — verify only)
        timedatectl
        # To change later:
        # sudo timedatectl set-timezone America/New_York

        # Hostname is set via user-data — DO NOT edit /etc/hostname directly
        # cloud-init will overwrite manual edits on next reboot
        # To rename: edit hostname: in /media/$USER/bootfs/user-data and reboot

        # Reboot and verify everything comes up clean
        sudo reboot
```

What NOT to do post-boot:

- Do NOT `sudo systemctl disable avahi-daemon` — breaks `.local` resolution
- Do NOT edit `/etc/hostname` directly — cloud-init overwrites on reboot
- Do NOT add `dhcpcd.conf` — Trixie uses NetworkManager
- Do NOT add `wpa_supplicant.conf` — conflicts with NetworkManager

Quick Reference — Connection Checklist

After full setup, standard connection workflow:

```
        # 1. Bring up WireGuard tunnel (from anywhere)
        sudo wg-quick up rpi-tunnel

        # 2. SSH in via tunnel
        ssh -i ~/.ssh/rpi_ed25519 -p 2222 pi@10.0.0.1

        # 3. Verify MAC on Pi
        ip link show wlan0 | grep ether
        # Expected: D2:AD:B3:3F:BE:EF

        # 4. Check WireGuard peers
        sudo wg show

        # 5. Tear down when done
        sudo wg-quick down rpi-tunnel
```

Vim — Survival Commands

> Full reference in Operator Handbook. These are the only commands needed in this guide.

```
i        → Insert mode (start typing)
Esc      → Back to Normal mode
:wq      → Save and quit
:q!      → Quit without saving
G        → Jump to bottom of file
ggdG     → Delete all content
/word    → Search
u        → Undo
```

APPENDIX — Legacy WiFi Configuration (Pre-Bookworm Only)

> **Do not use on Trixie.** This method is for Raspberry Pi OS Bullseye (Debian 11) and older only. On Trixie, `wpa_supplicant.conf` on the boot partition is not reliably processed — NetworkManager does not read it directly. Use Phase 4 `network-config` instead.

wpa_supplicant.conf (Bullseye and older)

```
        sudo vim /media/$USER/bootfs/wpa_supplicant.conf
```

```
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
update_config=1
country=US

network={
    ssid="YOUR_WIFI_SSID"
    psk="YOUR_WIFI_PASSWORD"
    key_mgmt=WPA-PSK
    proto=WPA RSN
}
```

> Do not use alongside NetworkManager — they conflict over `wlan0`. Do not combine with `network-config` or Imager WiFi settings. Pick one WiFi configuration path and use only that.

APPENDIX — Complete user-data Reference

> Same content as Phase 4 Step 3, compacted to fit one page for quick copy/print.

```
#cloud-config
manage_resolv_conf: false

# Hostname — only change via user-data on Trixie
# raspi-config changes are overwritten by cloud-init on reboot
hostname: <your-hostname>
manage_etc_hosts: true
preserve_hostname: false

timezone: America/New_York
keyboard:
  model: pc105
  layout: "us"

# Pi has no RTC — apt fails before NTP sync without this
apt:
  preserve_sources_list: true
  conf: |
    Acquire {
      Check-Date "false";
    };

packages:
  - avahi-daemon
  - vim
  - wireguard
  - macchanger
  - watchdog
  - ufw                # firewall (conflicts with iptables-persistent — pick one)
  - fail2ban
  - iptables
  - nmap
  - net-tools
  - git
  - unzip
  - tar
  - jq
  - tree
  - tmux

users:
  - name: pi
    groups: users,adm,dialout,audio,netdev,video,plugdev,cdrom,games,input,gpio,spi,i2c,render,sudo
    shell: /bin/bash
    lock_passwd: false     # REQUIRED — default is true which locks account even with hash set
    passwd: '<paste-$6$-hash-here>'

# enable_ssh is a cloudinit-rpi extension — works on Pi OS Trixie only
enable_ssh: true
ssh_pwauth: true

# Pi-specific interfaces (cloudinit-rpi module)
rpi:
  interfaces:
    serial: true

write_files:
  # MAC spoof via systemd-networkd .link file
  # Applied by udev when wlan0 is first detected — BEFORE NetworkManager
  # This is the only way to spoof MAC before WiFi association
  - path: /etc/systemd/network/00-wlan0-mac.link
    permissions: '0644'
    owner: root:root
    content: |
      [Match]
      OriginalName=wlan0

      [Link]
      MACAddress=D2:AD:B3:3F:BE:EF
      NamePolicy=keep kernel database onboard slot path

  # Belt-and-suspenders backup service if .link file doesn't apply
  - path: /etc/systemd/system/macspoof-wlan0.service
    permissions: '0644'
    owner: root:root
    content: |
      [Unit]
      Description=MAC spoof backup (if .link file failed)
      Wants=network-pre.target
      Before=network-pre.target NetworkManager.service
      After=sys-subsystem-net-devices-wlan0.device
      BindsTo=sys-subsystem-net-devices-wlan0.device
      DefaultDependencies=no
      ConditionFileNotEmpty=!/sys/class/net/wlan0/address

      [Service]
      Type=oneshot
      RemainAfterExit=yes
      ExecStartPre=/usr/sbin/ip link set dev wlan0 down
      ExecStart=/usr/bin/macchanger --mac=D2:AD:B3:3F:BE:EF wlan0
      ExecStartPost=/usr/sbin/ip link set dev wlan0 up

      [Install]
      WantedBy=network-pre.target

runcmd:
  # CRITICAL: Pi has no RTC. On first boot it thinks it's the image build date.
  # Modern apt uses sqv (Sequoia GPG) which rejects signatures dated in the future.
  # Force NTP sync BEFORE package install or apt fails with "Not live until..." errors.
  - [ sh, -c, "timedatectl set-ntp true" ]
  - [ sh, -c, "until timedatectl status | grep -q 'System clock synchronized: yes'; do sleep 2; done" ]
  # Now safe to install packages
  - [ sh, -c, "apt-get update && apt-get install -y avahi-daemon vim wireguard macchanger watchdog ufw fail2ban iptables nmap net-tools git unzip tar jq tree tmux" ]
  # crda deprecated on Trixie — cmdline.txt handles regdomain — sed harmless if absent
  - [ sh, -c, "sed -i 's/^[[:space:]]*REGDOMAIN=.*/REGDOMAIN=US/' /etc/default/crda 2>/dev/null || true" ]
  - [ systemctl, daemon-reload ]
  - [ systemctl, enable, macspoof-wlan0.service ]
  - [ sh, -c, "systemctl enable --now ssh || true" ]
```

APPENDIX — Time-Correlation Attacks & NTP Defenses

> Reference: how synchronized clocks create forensic evidence and how operational security practices use NTP obfuscation to defend against timeline reconstruction.

```
THE CORE CONCEPT
Every device logs timestamps. Investigators chain devices together by aligning logs across systems. Timing IS the fingerprint.

Example reconstructable timeline:
  Coffee shop WiFi    14:30:20   → DHCP lease to laptop MAC
  Laptop syslog       14:30:22   → SSH session opened
  WireGuard server    14:30:21   → Tunnel established from public IP
  Pi journal          14:30:22   → SSH accepted on port 2222
  Target server       14:30:25   → Login from VPN exit IP

If NTP-synced: align within seconds = strong correlation
If drifted:    require manual offset analysis = weak correlation

_____________________________________________________________________

THE NTP PARADOX
NTP solves real problems (apt cert validation, log accuracy, scheduled jobs) but enables forensic chaining.
The same property that makes systems RELIABLE makes timelines RECONSTRUCTABLE.
This is the central tension in operational security: convenience vs metadata obfuscation.

Without NTP sync:                  With NTP sync (default):
  Laptop:  14:30:22                  Laptop:  14:30:22
  Pi:      13:15:08 (drifted)        Pi:      14:30:22
  Target:  14:30:21                  Target:  14:30:21
  Result: weak correlation           Result: strong correlation

_____________________________________________________________________

MARCUS HUTCHINS CASE (2017)
British security researcher who stopped WannaCry ransomware was later charged with creating Kronos banking malware.
Prosecutors built timeline evidence correlating IRC logs, forum posts, code commits, Bitcoin transactions, ISP records.
Aligned timestamps across platforms argued for keyboard presence at specific moments.
Pleaded guilty 2019, sentenced to time served. Demonstrates how timestamp correlation builds presence evidence.

_____________________________________________________________________

STAGGERED NTP DEFENSE — TEMPORAL OBFUSCATION
Obfuscation goal: make automated timeline correlation expensive enough that investigators move to other evidence sources.

Standard (vulnerable):              Staggered (basic hardening):
  All devices → time.cloudflare       Laptop → time.nist.gov
                                      Pi     → time.cloudflare.com
                                      Phone  → time.apple.com
  Clocks: milliseconds aligned        Clocks: 50-500ms drift
  Easy automated correlation          Manual offset analysis required

This doesn't HIDE time — it defeats AUTOMATED correlation tools. Investigators can still align manually but friction slows analysis.

_____________________________________________________________________

LAYERED OPSEC DEFENSE STACK
Layer 1  Temporal obfuscation       different NTP time sources per device (basic)
Layer 2  Controlled drift           local NTP server with coordinated drift pattern (advanced)
Layer 3  Network obfuscation        Tor variable delays, mixnet jitter, traffic shaping
Layer 4  Behavioral obfuscation     vary timing of sensitive actions, no predictable rhythms
Layer 5  Metadata hygiene           MAC randomization, hostname rotation, log scrubbing

_____________________________________________________________________

CONFIGURE STAGGERED NTP ON LINUX

# Check current NTP source
timedatectl show-timesync --all | grep -i server

# Change NTP source
sudo vim /etc/systemd/timesyncd.conf
  Edit:  NTP=time.nist.gov                ← different from other devices
         FallbackNTP=pool.ntp.org

# Apply changes
sudo systemctl restart systemd-timesyncd
sudo timedatectl status                   # verify "System clock synchronized: yes"

_____________________________________________________________________

PUBLIC NTP SOURCES                  SUGGESTED PER-DEVICE ASSIGNMENT
time.nist.gov          US NIST        Workstation/laptop  → time.nist.gov
time.cloudflare.com    Cloudflare     Pi/server           → time.cloudflare.com
time.apple.com         Apple          Mobile device       → time.apple.com
time.windows.com       Microsoft      Router              → pool.ntp.org
time.google.com        Google (smeared)
pool.ntp.org           Community pool

_____________________________________________________________________

OPSEC KEY TAKEAWAY
NTP sync is REQUIRED for system functionality but CREATES forensic evidence.
Obfuscation defense is NOT disabling NTP — it's choosing different sources per device so automated timeline reconstruction tools fail.
For most users: single-source NTP is fine.
For operations needing defense against timeline analysis: staggered sources are the minimum opsec baseline.
Combines with other metadata hygiene practices (MAC obfuscation, traffic encryption, hostname randomization) for layered defense.
```